

Question 20

Q: An application requires detecting fine details in images. Explain how sampling rate and intensity resolution should be chosen to meet this requirement.

Theoretical Concept: Detecting Fine Details

To detect microscopic or fine details (like cracks in a microchip):

1. **Sampling Rate:** Must be extremely high (High Spatial Resolution) so that the crack covers several pixels.
2. **Intensity Resolution:** Must be high (e.g., 10-bit or 12-bit) if the defect has a very similar color/brightness to the background material.

OpenCV Python Solution

 Copy

```
import cv2 # Import OpenCV for image processing algorithms
import numpy as np # Import Numpy for mathematical matrix operations
import matplotlib.pyplot as plt # Import Matplotlib to visualize the output on screen

# Step 1: Simulate a high spatial resolution capture of a flat material
img = np.full((100, 100), 100, dtype=np.uint8) # Create a solid gray image array filled with a value

# Step 2: Draw an extremely faint, 1-pixel wide defect (e.g. micro-crack)
```

```
cv2.line(img, (20, 20), (80, 80), 90, 1) # Draw a straight line on the image

# Step 3: Apply Canny Edge Detection to extract the high-frequency defect
# Adjusting thresholds allows extraction of even the faintest anomalies
edges = cv2.Canny(img, threshold1=80, threshold2=120) # Detect edges using the Canny edge detection algorithm

# Step 4: Display the automated defect detection
plt.figure(figsize=(8,4)) # Create a new figure window for display
plt.subplot(121), plt.imshow(img, cmap='gray', vmin=0, vmax=255), plt.title('Raw Capture') # Place this image in a
plt.subplot(122), plt.imshow(edges, cmap='gray'), plt.title('Defect Extracted (Canny)') # Place this image in a su
plt.show() # Show the final plot window to the user
```

Expected Output

The raw image looks almost entirely flat and empty. The edge detection plot reveals a stark, bright diagonal line, proving how spatial filters isolate critical microscopic details from background noise.

How to Execute & Submit

1. Google Colab Execution:

- Open [Google Colab](#) and create a New Notebook.
- Click the  **Copy** button on the code block above.
- Paste it into a Colab cell and press **Shift + Enter** to run.

2. Uploading to GitHub:

- In Colab, go to **File > Save a copy in GitHub**.
- Authorize your GitHub account.
- Select your Computer Vision Lab repository and click **OK** to commit the code.